

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

C03: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks: 50

CASE STUDY:

1 A multinational software company develops a large ERP application consisting of 120 dynamically linked libraries. During execution, users complain that startup time is high while memory utilization remains low. As the system architect, analyze whether static linking or dynamic linking is the better choice. Recommend an optimized loading strategy by considering compile time, load time, execution time, and shared libraries. Justify your decision with suitable memory diagrams.

2 A cloud server simultaneously executes AI applications, database services, and web applications. Although 128 GB RAM is installed, memory allocation failures occur frequently. Investigate the causes by analyzing dynamic storage management, external fragmentation, and internal fragmentation. Design a memory management strategy that minimizes memory wastage while maintaining high throughput.

3 An airline reservation system experiences severe delays whenever thousands of customers book tickets simultaneously. The operating system frequently swaps processes between RAM and disk. Analyze how swapping affects CPU utilization and response time. Propose modifications to reduce swapping overhead without upgrading hardware.

4 A hospital management system allocates memory using contiguous memory allocation. After months of continuous operation, several large applications fail to load despite

sufficient free memory. Investigate the reason using fragmentation concepts. Compare First Fit, Best Fit, Worst Fit, and Next Fit strategies, and recommend the most appropriate allocation technique for this scenario.

5 A banking application initially uses contiguous memory allocation but is later migrated to paging. Compare the performance of both approaches under heavy transaction loads. Evaluate page table overhead, fragmentation, memory utilization, and execution efficiency before recommending the migration strategy.

6 A gaming company develops an open-world game with multiple independent modules such as graphics, AI, physics engine, and audio processing. Analyze how segmentation can improve memory organization compared to paging. Design an appropriate segment table and explain how logical addresses are translated into physical addresses.

7 A software development company uses both segmentation and paging in its operating system. During debugging, developers notice increased address translation time. Analyze the advantages and disadvantages of segmentation with paging compared to pure paging and pure segmentation. Recommend an optimized architecture for the organization.

8 A mobile operating system allows users to run multiple applications simultaneously, exceeding the available physical memory. Analyze how virtual memory enables efficient multitasking. Explain the consequences if virtual memory is disabled, and propose improvements for better memory performance.

9 An online video editing application loads only required portions of videos into memory. Users report occasional delays when editing large files. Analyze how demand paging influences system performance. Suggest methods to reduce page faults without increasing RAM capacity.

10 A data analytics server experiences frequent page faults while processing large datasets. After monitoring memory references, the administrator notices repetitive loading and replacement of the same pages. Analyze the effectiveness of FIFO, LRU, Optimal, and Clock page replacement algorithms. Recommend the most suitable algorithm for this workload with proper justification.

11 A university examination server crashes during online examinations due to excessive page faults. Investigate whether thrashing is responsible for the failure. Analyze CPU

utilization, page fault frequency, and working set size. Design an improved memory allocation policy to eliminate thrashing.

- 12 A cloud provider hosts hundreds of virtual machines on a single physical server. During peak hours, several VMs become unresponsive. Analyze the role of working sets in preventing thrashing. Propose a memory allocation framework that dynamically adjusts working set sizes based on workload patterns.

- 13 A multimedia company stores high-definition video files on HDDs while frequently accessed metadata is stored on SSDs. Analyze how disk devices influence virtual memory performance, swapping speed, and page replacement efficiency. Recommend a storage hierarchy that minimizes access latency.

- 14 A software company plans to redesign its operating system memory manager. The proposed system integrates dynamic linking, demand paging, segmentation, virtual memory, and SSD-based swapping. Evaluate whether this integrated approach improves overall system performance. Identify possible bottlenecks and propose architectural improvements.

- 15 An e-commerce platform experiences inconsistent response times during festival sales. System logs reveal excessive dynamic library loading, frequent page replacements, and swapping activity. Analyze the interaction among dynamic linking, demand paging, and swapping. Develop a comprehensive optimization strategy that improves performance while minimizing memory consumption.

- 16 A research laboratory develops a scientific simulation requiring large matrices exceeding available RAM. Compare the effectiveness of virtual memory, segmentation, paging, and segmentation with paging. Recommend the most efficient memory management model by considering scalability, performance, and memory protection.

- 17 A cybersecurity organization requires strict memory protection between user processes and kernel processes. Analyze how segmentation, paging, and segmentation with paging provide isolation and protection. Design a secure memory architecture capable of preventing unauthorized memory access while maintaining execution efficiency.

- 18 A fintech company is designing a next-generation operating system for financial servers. The system must support secure memory allocation, efficient page

replacement, fast loading of shared libraries, and minimal disk I/O. Develop a complete memory management framework integrating dynamic storage management, paging, virtual memory, working sets, and SSD storage. Justify each design decision.

19 An autonomous vehicle runs perception, navigation, and object detection modules concurrently on limited embedded memory. Analyze how demand paging, page replacement policies, virtual memory, and working set models influence real-time performance. Recommend a customized memory management strategy that guarantees deterministic execution with minimal latency.

20 A technology company plans to build an operating system for future AI-powered supercomputers. The system must efficiently support millions of concurrent processes, shared libraries, heterogeneous memory devices, SSDs, and distributed storage. Design a complete memory management architecture integrating linkers, dynamic linking, dynamic storage management, swapping, contiguous allocation, paging, segmentation, segmentation with paging, virtual memory, demand paging, page replacement, thrashing prevention, working sets, and advanced disk management. Evaluate trade-offs among performance, scalability, reliability, and cost.

Advanced Operating Systems & Memory Management Architecture

Comprehensive Technical Analysis, Algorithmic Evaluation, and Systems Optimization

1. ERP Application Loading Strategy

Linking Trade-Off Analysis

Metrics / Parameters	Static Linking	Standard Dynamic Linking	Deferred Dynamic Loading
Compile Time	High (link step resolves 120 libraries into 1 binary).	Fast (links only import stubs).	Fast (links stubs / no build-time linking).
Load Time	Extremely Fast (OS loads binary directly).	High (Dynamic linker resolves all 120 DLLs/SOs at startup).	Minimal (Only core libraries load at startup).
Execution Time	Fast (Direct jumps, no PLT/GOT overhead).	Slight indirect call overhead via PLT/GOT.	Direct call after runtime binding.
Shared Memory	Zero sharing across processes (Code duplication).	Shared text segment among running processes.	Shared text segment, loaded on demand.

Architectural Recommendation

The high startup latency occurs because the dynamic linker (ld.so / dyld) resolves dependencies and performs symbol relocations across all 120 libraries at startup before executing main().

Recommended Strategy: Hybrid Dynamic Linking with Lazy Loading

1. Core Runtime (Static/Eager Dynamic): Link ~10–15 essential libraries (UI shell, core

security, logging) eagerly at startup.

2. Feature Modules (Lazy/Deferred Dynamic): Convert remaining 105+ domain-specific libraries (HR, Inventory, Finance) into dynamically loaded plugins using `dlopen()` / `FreeLibrary()`.

2. Dynamic Storage Management for Cloud Workloads

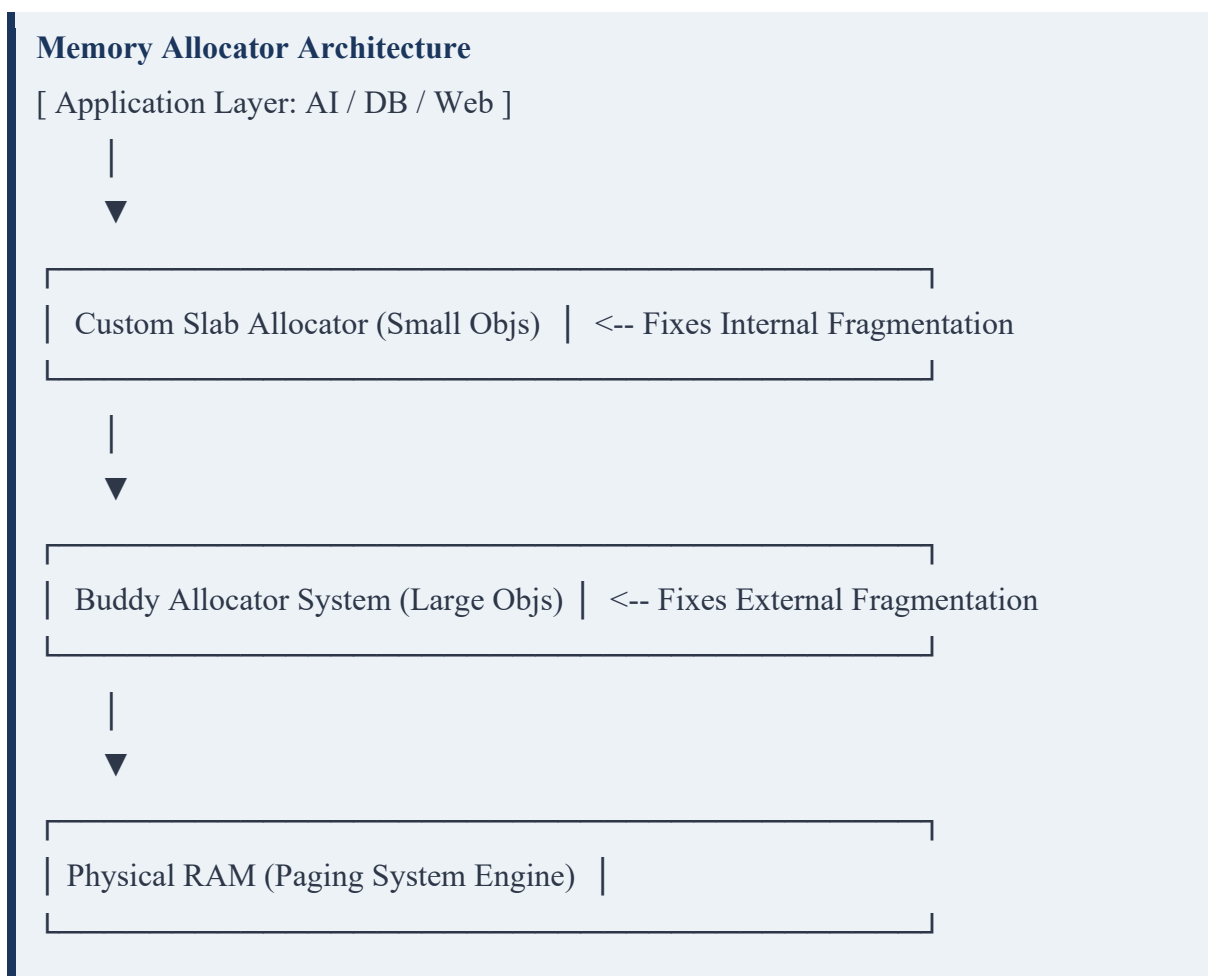
Causes of Memory Allocation Failures

Despite 128 GB RAM installed, allocation requests fail due to two primary fragmentation issues:

1. External Fragmentation: Free memory is split into numerous non-contiguous blocks. An allocation request for a contiguous 16 MB block fails if the largest contiguous free chunk is 8 MB, even if total free RAM exceeds 30 GB.

2. Internal Fragmentation: Occurs when fixed-size allocation systems round up requests (e.g., allocating a 4 KB page for a 100-byte object), wasting trailing space within allocated regions.

Strategy: Hybrid Memory Architecture



- **Buddy Allocator System:** Allocates memory in powers-of-two pages. Merges adjacent free blocks ('buddies') during deallocation to restore contiguous space.
- **Slab Allocator Layer:** Sub-allocates small, frequently created objects (e.g., database handles, web request sockets) from buddy pages without triggering kernel-level fragmentation.

3. Airline Reservation System Swapping Overhead

Swapping Impact Analysis

Under surge conditions, thousands of concurrent processes compete for physical pages, forcing the OS page daemon into rapid swap-out and swap-in operations (disk thrashing).

- **CPU Utilization:** Drops toward zero as CPU cycles are spent waiting for disk I/O interrupts.
- **Response Time:** Degrades exponentially (from milliseconds to tens of seconds) due to mechanical or SSD latency bottlenecks during page reads/writes.

Optimization Strategy (Zero-Hardware Cost)

Proposed	System	Modifications
1. Replace Process-per-Request with Thread Pooling:	Transition from fork-based models to an asynchronous event loop (e.g., EPoll/NIO) with worker threads matched to physical CPU	cores.
2. zRAM / zswap Compression:	Compress evicted pages in RAM instead of issuing synchronous disk I/O	writes.
3. Set Memory Limits & Sticky Sessions:	Establish explicit heap boundaries to prevent process swapping.	

4. Contiguous Allocation Analysis for Hospital Systems

Investigation

Over months of continuous operation, dynamic process spawning and termination cause external fragmentation. Free memory becomes scattered into small holes. A large application requiring 200 MB contiguous space fails if the largest available free block is only 120 MB, even if total free space exceeds 2 GB.

Strategy Comparison

Strategy	Search Overhead	Fragmentation Pattern	Scenario Suitability
First Fit	Low (stops at first valid block).	Creates small unusable holes at the start of memory.	General execution.
Best Fit	High (scans entire free list).	Produces tiny, unusable residual fragments.	Poor for dynamic workloads.
Worst Fit	High (scans entire free list).	Leaves large residual blocks, but wastes large free space quickly.	Suboptimal overall.

Next Fit	Low (resumes from last search point).	Spreads fragments throughout entire memory address space.	High allocation churn.
----------	---------------------------------------	---	------------------------

Recommendation: Use Best Fit with Compaction (or Dynamic Relocation using Base/Limit registers) in the short term, but transition from contiguous allocation to Paging long-term to completely eliminate external fragmentation.

5. Banking Application: Contiguous Allocation vs. Paging

Comparative Performance Metrics

Metric	Contiguous Allocation	Paging Model
Page Table Overhead	None (uses single Base/Limit register pair).	High (4 KB - 4 MB per process depending on depth).
Fragmentation	Severe External Fragmentation. Zero Internal.	Zero External Fragmentation. Minor Internal (last page).
Memory Utilization	Low (suffers from memory lockup & rejection).	Near 100% efficiency for active processes.
Execution Speed	Fast translation (Physical = Base + Logical).	Relies on TLB hardware caching for table walks.

Migration Decision: Migrate to Paging with HugePages (2 MB / 1 GB pages for core database buffers). While multi-level page table lookups add small translation overhead, modern CPUs mitigate this with hardware TLBs. Paging prevents transaction drops caused by allocation failures under heavy load.

6. Open-World Game Memory Management with Segmentation

Segmentation vs. Paging for Game Engine Modules

Games are structured logically around modular code and data spaces (Graphics, Physics Engine, Audio, AI). Paging breaks code arbitrarily into fixed 4 KB frames, whereas Segmentation maps segments directly to functional modules with tailored protection bits (e.g., Read-Only for Audio assets, Read-Write for Physics data).

Sample Game Segment Table

Segment ID	Module Name	Base Address	Limit (Size)	Permissions
0x0	Core Engine Code	0x00040000	12 MB	Read / Execute
0x1	Physics State	0x00CC0000	64 MB	Read / Write
0x2	Audio Buffers	0x04CC0000	128 MB	Read / Write
0x3	Graphics Textures	0x0CCC0000	512 MB	Read-Only

7. Hybrid Memory: Segmentation with Paging

Architectural Trade-Off Analysis

Memory Model	External Frag.	Internal Frag.	Address Translation Overhead	Protection & Sharing
Pure Paging	None	Minor (last frame)	Single Table / Multi-level Walk	Low (page granularity)
Pure Segmentation	High	None	Low (Segment Table lookup)	High (logical boundary)

Segmentation with Paging	None	Minor (last page)	Two Lookups (Segment + Page Table)	Maximum (logical + frame level)
--------------------------	------	-------------------	------------------------------------	---------------------------------

Recommendation: Adopt Paged Architecture with Segment Registers (flat-memory paging model). Logical protection is enforced using software attributes inside multi-level page tables (e.g., NX-bit, User/Supervisor flags), restoring hardware-accelerated TLB lookups.

8. Mobile Operating System Virtual Memory Design

Virtual Memory provides each app with an isolated, contiguous address space larger than available RAM by using page mappings.

Impact of Disabling VM: If Virtual Memory is disabled, applications run directly in physical RAM, causing immediate crash-on-launch under multitasking and risking cross-process memory corruption.

Optimization Plan: Implement Compressed In-Memory Paging (zswap) to compress background app pages in RAM, and employ a Low Memory Killer (LMK) daemon to terminate low-priority cached processes during memory pressure.

9. Demand Paging in Video Editing Software

Demand paging loads video frame chunks into RAM only when referenced. Delay is driven by Page Fault Interrupts during scrubbing.

Non-Hardware Optimization Techniques

1. Prefetching / Read-Ahead: Predict playback direction and speculatively load the next N frames into page memory.
2. Sequential Memory Mapping (madvise): Issue `madvise(MADV_SEQUENTIAL)` hints to aggressively prefetch contiguous disk blocks.
3. Clustered Page Fault Processing: Resolve page faults in batches (64 KB clusters) to reduce I/O context-switch overhead.

10. Data Analytics Server Page Replacement Strategy

Algorithm	Performance / Page Fault Ratio	Implementation Complexity	Real-World Overhead
FIFO	Poor (Belady's Anomaly vulnerability).	Very Low (Queue).	Ignores frequency/recency of page usage.
LRU	Optimal Approximation.	High (Requires stack/timestamp tracking).	High hardware/runtime tracking cost.
Optimal (OPT)	Lowest Possible Faults (Theoretical baseline).	Unachievable in real systems.	Requires future reference knowledge.
Clock (Second Chance)	Approximates LRU effectively.	Low (Single Reference Bit + Pointer Ring).	Low overhead; ideal for production OS.

Recommendation: Implement Clock with Adaptive Reference Bit (or CAR / LRU-K). Full LRU tracking adds high tracking cost, whereas Clock provides near-LRU efficiency with $O(1)$ replacement overhead.

11. University Server Thrashing Analysis

During peak exam load, context switches spike concurrent execution, making total Working Set Size (WSS) exceed available physical memory. The system enters thrashing: pages are continuously swapped out and in, causing CPU utilization to plummet.

Mitigation Policy

- Working Set Model Guard: Track WSS for each process over a sliding window. Suspend lower-priority processes if $\text{sum}(\text{WSS}) > \text{Total RAM}$.

- **Page Fault Frequency (PFF) Controls:** Dynamically allocate frames to processes exceeding upper PFF thresholds and swap out processes that cannot obtain sufficient frames.

12. Cloud Virtual Machine Working Set Framework

To manage competing VM workloads on a single hypervisor host:

- 1. Memory Ballooning:** A hypervisor balloon driver running inside each guest OS inflates to reclaim unused guest pages when another VM requires additional memory.
- 2. Kernel Samepage Merging (KSM):** Scan physical memory to merge duplicate memory pages (e.g., identical OS kernel pages across guest VMs) into single Copy-on-Write (CoW) pages.
- 3. Adaptive Allocation:** Periodically sample access bits across VM memory regions to readjust frame allocations dynamically.

13. Storage Hierarchy for HDDs, SSDs, and RAM

Storage Type	Sequential R/W	Random 4K Read IOPS	Latency	Virtual Memory Role
HDD	Medium (150 MB/s)	Low (~150 IOPS)	High (10 ms)	Long-term cold data storage.
NVMe SSD	High (5000 MB/s)	High (~500,000 IOPS)	Low (50 μ s)	Fast swap space / metadata store.
RAM	Extremely High (50 GB/s)	Near Unlimited	Extremely Low (50 ns)	Active page frames / working sets.

Optimized Hierarchy: Place swap partitions exclusively on NVMe SSD storage and cache index nodes, directory tables, and video file headers in RAM/SSD to eliminate HDD random seek penalties.

14. Integrated OS Memory Manager Assessment

Combining Dynamic Linking, Demand Paging, Segmentation, Virtual Memory, and SSD Swapping creates high flexibility but introduces potential bottlenecks:

- **Double Lookup Overhead:** Resolving segments then pages increases translation latency. Fix: Use flat-mode segmentation with single-level hardware-accelerated TLB caching.
- **SSD Bus Saturation:** High page-fault rates degrade SSD lifespan and saturate PCIe lanes. Fix: Implement an in-memory compressed RAM cache (zswap) before SSD swapping.
- **Dynamic Linking Latency:** Delayed symbol resolution causes runtime stalls. Fix: Pre-bind frequently used symbols at startup (LD_BIND_NOW).

15. E-Commerce Platform Optimization Under Flash Sales

Comprehensive Optimization Strategy

Step 1: Force eager dynamic binding at startup (LD_BIND_NOW=1) to eliminate symbol lookups during peak sales.

Step 2: Lock critical transaction processing code in RAM using mlockall() to prevent swapping.

Step 3: Enable zswap to compress evicted non-critical pages in RAM, reducing SSD I/O stalls during transaction spikes.

16. Matrix Simulation Memory Model Comparison

Memory Model	Matrix Traversal Scaling	Cache Line Utilization	Memory Protection	Scalability
Virtual Memory (Paging)	High efficiency for row-major sweeps.	Good (aligned to page boundaries).	Page-level RW bits.	Excellent.
Pure Segmentation	Fails when matrix exceeds segment limit.	Poor across boundaries.	Segment-level limits.	Poor.
Segmentation with Paging	High protection overhead.	Mediocre due to double translation.	Dual-layer protection.	Moderate.

Recommendation: Use Pure Virtual Memory with 1 GB HugePages and Row-Major Storage Alignment. HugePages minimize page table entry overhead and eliminate TLB misses across multi-gigabyte matrix traversals.

17. Secure Kernel/User Isolation Architecture

- 1. Paging Privilege Levels:** Use Page Table Entry protection bits (User/Supervisor, Read/Write, No-Execute bits).
- 2. Kernel Page Table Isolation (KPTI):** Separate user and kernel page tables to prevent speculative side-channel attacks (e.g., Meltdown).
- 3. Hardware Protection Flags:** Enable Supervisor Mode Execution Prevention (SMEP) and Supervisor Mode Access Prevention (SMAP) to block unauthorized execution or access of user memory by kernel code.

18. Fintech Operating System Memory Manager Design

- **Allocation Layer:** Uses a Slab Allocator built over a Buddy System to eliminate external fragmentation and provide zero-latency memory allocation for small transaction objects.
- **Paging & Virtual Memory:** Standard 4 KB pages for application logic paired with 2 MB HugePages for in-memory database buffers.
- **Shared Libraries:** Shared library code is mapped into process address spaces as read-only, copy-on-write segments.
- **Page Replacement:** Uses Clock-Pro algorithm to distinguish between frequently and recently accessed pages.
- **Storage Hierarchy:** Primary execution in RAM, backed by zswap compression and enterprise NVMe SSD swap.

19. Real-Time Autonomous Vehicle Memory System

Standard Virtual Memory with dynamic demand paging introduces unpredictable 50 μ s–10 ms latency penalties, making it unsuitable for real-time safety systems.

Real-Time Memory Blueprint

1. **Disable Demand Paging Post-Boot:** Pre-allocate and map all memory pages for safety-critical modules (Perception, Actuation) during boot time.
2. **Page Locking (mlockall):** Lock core module execution segments into physical RAM to prevent eviction.
3. **Partitioned Static Working Sets:** Allocate fixed, dedicated physical RAM pools to critical processes.

20. Supercomputer OS Memory Architecture

1. **Heterogeneous Memory Manager (HMM):** Unified address space mapping host DDR5 RAM, High-Bandwidth Memory (HBM3), and GPU/NPU VRAM using mirrored page tables.
2. **Dynamic Linker Subsystem:** Performs global symbol pre-binding during app initialization, caching dynamic link resolution across execution nodes.
3. **Variable Page Size Engine:** Uses 4 KB pages for control tasks, 2 MB pages for worker threads, and 1 GB pages for large AI model weights.

4. Thrashing Prevention: Monitors process Working Set Sizes across compute nodes and migrates execution using RDMA if pressure rises.

Architectural Trade-Off Analysis

Metric	System Design Choice	Trade-Off & Mitigation Strategy
Performance vs. Memory Cost	Aggressive page pre-allocation & HBM3 utilization.	High hardware costs; mitigated by tiered offloading to NVMe arrays.
Scalability vs. Overhead	Multi-level hierarchical page tables with migration.	Increased page-walk latency; mitigated by heavy hardware TLB caching.
Reliability vs. Latency	Synchronous memory check-pointing to NVDIMMs.	Minor write performance impact; yields complete crash recovery capabilities.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Case	25	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand